



Introduzione ad Agda: programmazione funzionale con tipizzazione dipendente

Università di Verona
Imbriani Paolo - VR500437

6 novembre 2025

Indice

1	Introduzione	4
1.1	Che cos'è la Teoria dei Tipi?	4
1.2	Cos'è Agda?	4
1.2.1	Caratteristiche principali di Agda	5
2	Primi passi con Agda	5
2.1	Tipi di base	5
2.2	Definizione di operatori	6
2.3	Pattern matching e Goals	6
2.4	Liste	7
2.5	Maybe	8
3	Uguaglianza in Agda	9
3.1	Definizione di uguaglianza	9
3.2	Congruenza	11
3.3	Simmetria	12
3.4	Transitività	12
3.5	Associatività	12
3.6	Commutatività	13
4	Metalogica in Agda	14
4.1	Definizione delle proposizioni e formule	14
4.2	Definizione dei contesti	14
4.3	Definizione delle regole di deduzione naturale	15
4.4	Esempio di prova in Agda	16

Prefazione

Questa dispensa e raccolta di appunti è stata realizzata durante il tirocinio del terzo anno di Informatica (A.A 2025/2026) presso l'Università di Verona. L'obiettivo principale di questo lavoro è quello di aiutarmi nel corso del tirocinio per avere le conoscenze di base di Agda per poter modellare la logica modale S4.2 utilizzando come strumento proprio Agda. All'interno di questo documento vengono usati anche estratti di lezioni del mini-corso «*Introduction to Type Theory*» tenute dal professore *Thorsten Altenkirch*, docente di Informatica presso l'Università di Nottingham. Questa dispensa non vuole **sostituire** i testi ufficiali sulla teoria dei tipi o su Agda, ma vuole essere un **supporto** per chi si avvicina per la prima volta a questo linguaggio di programmazione funzionale con tipizzazione dipendente.

1 Introduzione

1.1 Che cos'è la Teoria dei Tipi?

Originariamente sviluppata come alternativa alla teoria degli insiemi (*Axiomatic Set Theory*) per fondare la matematica, la teoria dei tipi è ora ampiamente utilizzata in informatica, in particolare nei linguaggi di programmazione funzionale e nei sistemi di verifica formale. Non solo, molto semplicemente la teoria dei tipi la possiamo vedere come una logica costruttiva ed intuizionista.

Caratteristiche principali

- **Il typing come giudizio:**

$$3 \in \mathbb{N} \quad \text{o equivalentemente} \quad 3 : \mathbb{N}$$

Nella Teoria dei Tipi, il typing è un giudizio, non una proposizione. La Teoria dei Tipi è staticamente tipata — la correttezza dei tipi è verificata dal sistema stesso.

- **Tipi dipendenti:** I tipi possono dipendere dai valori. Prendiamo per esempio il tipo dei vettori:

$$\text{Vec} : \mathbb{N} \rightarrow \text{Type} \quad \text{dove} \quad \text{Vec}(n) \text{ è il tipo dei vettori di lunghezza } n.$$

- **Assioma di univalenza:** Identifica l'equivalenza di strutture con l'uguaglianza di tipi (come nella Homotopy Type Theory).
- **Operazioni insiemistiche:** Le usuali operazioni $\cup, \cap, \subseteq$ non sono definite direttamente sui tipi, ma sui sottoinsiemi (o predicati) di un dato tipo.
- **La logica è emergente:** La corrispondenza tra logica e tipi è nota come isomorfismo di Curry–Howard:

Proposizioni come Tipi, Dimostrazioni come Programmi.

- **Le funzioni sono primitive:** Le funzioni sono oggetti centrali nella Teoria dei Tipi. Ad esempio, una relazione R su tipi A, B può essere vista come:

$$R : A \rightarrow B \rightarrow \text{Prop}$$

1.2 Cos'è Agda?

Agda è un linguaggio di programmazione funzionale con tipizzazione dipendente e un sistema di dimostrazione interattivo. Sviluppato principalmente per la ricerca e l'insegnamento, Agda consente di scrivere programmi che sono anche prove formali, garantendo così la correttezza del codice attraverso il sistema di tipi.

1.2.1 Caratteristiche principali di Agda

- **Sintassi simile a Haskell:** Agda ha una sintassi che ricorda quella di Haskell, rendendolo accessibile a chi ha familiarità con i linguaggi funzionali.
- **Supporto Unicode:** Agda supporta caratteri Unicode, permettendo l'uso di simboli matematici direttamente nel codice.
- **Tipi dipendenti:** Agda supporta tipi che possono dipendere da valori, permettendo di esprimere proprietà più precise nei tipi stessi.
- **Propositions as Types (PaT):** In Agda, le proposizioni logiche sono rappresentate come tipi, e le prove di queste proposizioni sono rappresentate come programmi che abitano questi tipi.
- **Sistema di moduli:** Agda ha un sistema di moduli che consente di organizzare il codice in modo strutturato e riutilizzabile.
- **Interattività:** Agda offre un ambiente interattivo che facilita la scrittura e la verifica delle prove, con feedback immediato sullo stato delle dimostrazioni.

Agda ci permette di modellare concetti matematici e logici in modo rigoroso, fornendo un ponte tra teoria dei tipi, programmazione funzionale e logica formale. La cosa interessante è che funziona come se stessimo costruendo le fondamenta su cui poi poggeranno i nostri assiomi o dimostrazioni (è proprio così che funziona la logica costruttiva).

💡 Osservazione 1.1

Agda è un linguaggio di programmazione funzionale totale, il che significa che ogni funzione definita in Agda deve essere totale, ovvero deve essere definita per tutti gli input possibili e deve sempre terminare. Senza questa caratteristica, la logica alla base del linguaggio diventerebbe inconsistente, e sarebbe possibile dimostrare affermazioni arbitrarie.

2 Primi passi con Agda

2.1 Tipi di base

In agda, come già precedentemente detto, niente è realmente definito fin dall'inizio (se non dalle librerie standard di Agda). Tuttavia possiamo noi stessi definire i nostri tipi di base. Per esempio, definiamo il tipo dei numeri naturali, usando la notazione di Peano:

```
data ℕ : Set where
  zero : ℕ
  suc  : ℕ → ℕ
```

In questo tipo di dato, abbiamo due costruttori:

- zero rappresenta il numero naturale 0.
- suc è una funzione che prende un numero naturale n e restituisce il successore di n , cioè $n + 1$.

🔔 Osservazione 2.1

In Agda, tutto può essere un nome anche la stringa `134function_*3` poiché l'unico vincolo che abbiamo sui nomi è che non possono contenere spazi (poiché Agda usa gli spazi per definire i nomi).

2.2 Definizione di operatori

Possiamo definire operatori personalizzati in Agda per rendere il codice più leggibile. Ad esempio, definiamo l'operatore di somma per i numeri naturali:

```
_+_ : ℕ → ℕ → ℕ
zero + n = n
suc m + n = suc (m + n)
```

In questo esempio, l'operatore `+` è definito come una funzione che prende due numeri naturali e restituisce la loro somma. Altro esempio, definiamo l'operatore di moltiplicazione:

```
_*_ : ℕ → ℕ → ℕ
zero * n = zero
suc m * n = n + (m * n)
```

Solitamente, in Agda, per aggiungere chiarezza a questi operatori dobbiamo anche specificare la loro precedenza e associatività attraverso le direttive `infix` e `infixl/infixr`.

- `infixl n _op_`: Definisce l'operatore `op` come un operatore binario con associatività a sinistra e precedenza n .
- `infixr n _op_`: Definisce l'operatore `op` come un operatore binario con associatività a destra e precedenza n .
- `infix n _op_`: Definisce l'operatore `op` come un operatore binario non associativo con precedenza n .

Il valore n determina la precedenza dell'operatore, con valori più alti che indicano una precedenza più alta. Solitamente si usa una scala da 0 a 9 per la precedenza degli operatori.

2.3 Pattern matching e Goals

Agda aiuta nella scrittura delle funzioni tramite il **pattern matching**. Quando definiamo una funzione, possiamo specificare come essa si comporta per diversi casi basati sui valori di input.

Prendiamo come esempio la funzione somma che abbiamo scritto sopra, un modo per farci aiutare da Agda è quello di scrivere la firma della funzione e poi usare il comando **C-c C-l** per creare i **goals** (obiettivi) che dobbiamo soddisfare per completare la definizione della funzione.

```

_+_ : ℕ → ℕ → ℕ
_+_ = ?

```

Dopo aver premuto **C-c C-l**, Agda genererà i seguenti goals:

```

_+_ : ℕ → ℕ → ℕ
_+_ = {!0!}

```

Dove `{!0!}` rappresenta un buco che dobbiamo riempire per completare la definizione della funzione. Possiamo quindi completare la definizione della funzione somma utilizzando il pattern matching per specificare i casi. Basta inserire prima del simbolo `=` i parametri `m` e `n` e poi premere **C-c C-c** per generare i goals per ciascun caso:

```

_+_ : ℕ → ℕ → ℕ
zero + n = {!0!}
suc m + n = {!1!}

```

Ora, dobbiamo riempire i buchi con le espressioni appropriate per ciascun caso.

```

_+_ : ℕ → ℕ → ℕ
zero + n = {! n !}
suc m + n = {! suc (m + n) !}

```

Dopo aver completato la definizione, possiamo rimuovere i buchi facendo la combinazione di tasti **C-c C-SPACE** per verificare che la funzione sia correttamente definita. Alternativamente alla ridefinizione di ogni singolo concetto base, possiamo importare le librerie standard di Agda che contengono già definizioni di tipi e funzioni comuni.

2.4 Liste

Definiamo il tipo delle liste in Agda:

```

data List (A : Set) : Set where
  [] : List A
  _::_ : A → List A → List A

```

In questa definizione:

- `List` è un tipo parametrizzato da un tipo `A`, che rappresenta il tipo degli elementi della lista.

- `[]` rappresenta la lista vuota.
- `_ :: _` è un costruttore che prende un elemento di tipo A e una lista di tipo $List\ A$, restituendo una nuova lista con l'elemento aggiunto all'inizio.

Proviamo a definire la funzione `length` (per calcolare la lunghezza di una lista):

```
length : {A : Set} → List A → ℕ
length [] = zero
length (x :: xs) = suc (length xs)
```

In questa definizione:

- Prima di tutto, notiamo che la funzione `length` è parametrizzata da un tipo A (usando le parentesi graffe per indicare che è un parametro implicito e quindi non deve essere specificato esplicitamente quando si chiama la funzione).
- La funzione `length` prende una lista di tipo $List\ A$ e restituisce un numero naturale $\mathbb{N}$.
- Nel caso della lista vuota, la lunghezza è zero.
- Nel caso di una lista non vuota, la lunghezza è il successore della lunghezza della coda della lista.

Notiamo come il modo in cui definiamo le funzioni in Agda è in maniera ricorsiva e tramite pattern matching, che ci permette di definire il comportamento della funzione per diversi casi basati sui valori di input. Abbiamo un caso base (per la lista vuota) e un caso ricorsivo (per la lista non vuota).

2.5 *Maybe*

Per definire l'operatore di estrazione di un elemento da una lista, possiamo utilizzare il tipo `Maybe` per gestire il caso in cui l'elemento non esista.

```
data Maybe (A : Set) : Set where
  nothing : Maybe A
  just : A → Maybe A
```

In questa definizione:

- `Maybe` è un tipo parametrizzato da un tipo A , che rappresenta il tipo dell'elemento opzionale.
- `nothing` rappresenta l'assenza di un valore.
- `just` è un costruttore che prende un elemento di tipo A e restituisce un valore di tipo `Maybe A` che contiene quell'elemento.

Ora possiamo definire la funzione `_ !! _` per estrarre l' n -esimo elemento da una lista:

```

_!!_ : {A : Set} → List A → ℕ → Maybe A
[] !! n = nothing
(x :: xs) !! zero = just x
(x :: xs) !! (suc n) = xs !! n

```

In quest'altra definizione notiamo:

- La funzione `_!!_` è parametrizzata da un tipo A (usando le parentesi graffe per indicare che è un parametro implicito).
- La funzione prende una lista di tipo `List A` e un numero naturale $\mathbb{N}$ (l'indice dell'elemento da estrarre) e restituisce un valore di tipo `Maybe A`.
- Nel caso della lista vuota, restituisce `nothing`, indicando che non c'è nessun elemento da estrarre.
- Nel caso in cui l'indice sia zero, restituisce il primo elemento della lista avvolto in `just`.
- Nel caso in cui l'indice sia un successore, chiama ricorsivamente la funzione sulla coda della lista e sull'indice decrementato di uno.

Questo costrutto ci permette di gestire in modo sicuro l'estrazione di elementi da una lista, senza incorrere in errori di accesso a indici fuori dai limiti. Questa caratteristica è molto più comune in Haskell che in Agda, ma penso sia comunque interessante mostrarla.

🔔 Osservazione 2.2

Tuttavia, esiste una versione più sicura di questa funzione che sfrutta i tipi dipendenti che garantisce che l'indice sia sempre valido per ogni input, evitando di usare il tipo `Maybe`. È interessante notare come in Agda, con il giusto impegno e l'uso delle dimostrazioni formali, possiamo garantire la correttezza del nostro codice facendo in modo che *non cada mai in errori di runtime*.

3 Uguaglianza in Agda

In Agda, l'uguaglianza è un concetto fondamentale che viene trattato in modo rigoroso attraverso il sistema di tipi.

3.1 Definizione di uguaglianza

L'uguaglianza in Agda è definita tramite un tipo chiamato `_≡_`, che rappresenta l'uguaglianza tra due elementi di uno stesso tipo.

```
data _≡_ {A : Set} (x : A) : A → Set where
  refl : x ≡ x
```

In questa definizione:

- `_≡_` è un tipo parametrizzato da un tipo A e da un elemento x di tipo A .
- Il costruttore `refl` rappresenta il fatto che ogni elemento è uguale a se stesso.

Da qua, inizia tutto il lavoro di dimostrazione delle proprietà dell'uguaglianza. Infatti «`refl`» (che è nient'altro che la riflessività) è il blocchetto principale per poi costruire tutte le altre definizioni.

💡 Osservazione 3.1

Nella matematica tradizionale " $x = y$ " è un'affermazione, una dichiarazione. Ad esempio, questa affermazione potrebbe essere vera o falsa. In Agda invece, " $x \equiv y$ " è un tipo: il tipo dei testimoni che dimostrano che x e y sono uguali. Ad esempio, questo tipo potrebbe essere abitato (cioè esistere un valore di quel tipo), oppure potrebbe essere vuoto (cioè non esistere alcun valore di quel tipo). Quindi se nella matematica tradizionale dimostreremmo che $x = y$, in Agda esibiamo un valore di tipo $x \equiv y$.

Con la definizione di uguaglianza a disposizione, possiamo affermare e dimostrare che zero è uguale a zero:

```
zero-equals-zero : zero ≡ zero
zero-equals-zero = refl
```

In esattamente lo stesso modo, possiamo affermare e dimostrare che $2 + 2 = 4$:

```
grande-teorema : two + two ≡ four
grande-teorema = refl
```

Possiamo usare `refl` in esattamente quelle situazioni, in cui i due lati della presunta identità sono manifestamente uguali, cioè uguali in un modo che richiede solo lo srotolamento di tutte le definizioni e la semplificazione tramite il calcolo, ma nessuna vera intuizione, per convincersi dell'uguaglianza. Per esempio, essendo che $\text{zero} + b$ è definito come b , possiamo usare `refl` nella seguente dimostrazione:

```
trivial : (b : ℕ) → zero + b ≡ b
trivial b = refl
```

Questo tuttavia funziona solo perché abbiamo definito l'operatore di somma in un certo modo.

```

_+_ : ℕ → ℕ → ℕ
zero + n = n -- Guarda qui!
suc m + n = suc (m + n)

```

È tutt'altra storia il verso opposto dell'uguaglianza, cioè dimostrare che b è uguale a $zero + b$. In questo caso, non possiamo usare `refl`, perché b non è definito come $zero + b$. Dobbiamo prima però definire un paio di strumenti ausiliari.

3.2 Congruenza

La **congruenza** è una proprietà fondamentale dell'uguaglianza che afferma che se due elementi sono uguali, allora lo sono anche i risultati delle funzioni applicate a questi elementi. Proviamo a definire la funzione di congruenza in Agda:

```

cong : {A B : Set} {x y : A}
→ (f : A → B)
→ x ≡ y
→ f x ≡ f y
cong f refl = refl

```

In questa definizione:

- La funzione `cong` prende due tipi A e B , due elementi x e y di tipo A , una funzione f da A a B , e una prova che x è uguale a y .
- Restituisce una prova che $f(x)$ è uguale a $f(y)$.
- Il caso base della definizione afferma che se la prova di uguaglianza è data da `refl`, allora anche l'uguaglianza tra $f(x)$ e $f(y)$ è dimostrata da `refl`.

Ora possiamo effettivamente dimostrare che b è uguale a $zero + b$ usando la funzione di congruenza:

```

symm-zero-plus : (b : ℕ) → b + zero ≡ b
symm-zero-plus zero = refl
symm-zero-plus (suc n) = cong suc (symm-zero-plus n)

```

Vediamo come:

- Nel caso base, quando b è zero, la prova è semplicemente `refl`, poiché zero è uguale a $zero + zero$. (Stiamo praticamente usando la riflessività per dire che zero è uguale a zero).
- Nel caso ricorsivo, quando b è il successore di n , usiamo la funzione di congruenza per applicare la funzione `suc` alla prova che n è uguale a $zero + n$.

3.3 Simmetria

La **simmetria** è un'altra proprietà fondamentale dell'uguaglianza che afferma che se un elemento è uguale a un secondo, allora il secondo è uguale al primo. Proviamo a definire la funzione di simmetria in Agda:

```
sym : {A : Set} → {x y : A}
  → x ≡ y
  → y ≡ x
sym refl = refl
```

3.4 Transitività

La **transitività** è un'altra proprietà fondamentale dell'uguaglianza che afferma che se un elemento è uguale a un secondo, e il secondo è uguale a un terzo, allora il primo è uguale al terzo. Proviamo a definire la funzione di transitività in Agda:

```
trans : {A : Set} → {x y z : A}
  → x ≡ y
  → y ≡ z
  → x ≡ z
trans refl refl = refl
```

Ormai abbiamo capito che se la prova di uguaglianza è data da `refl`, allora possiamo concludere che l'uguaglianza tra x e z è dimostrata da `refl`.

3.5 Associatività

Possiamo anche dimostrare che l'operazione di somma è associativa utilizzando le proprietà dell'uguaglianza che abbiamo definito.

```
assoc-+ : (a b c : ℕ) → (a + b) + c ≡ a + (b + c)
assoc-+ zero b c = refl
assoc-+ (suc a) b c = cong suc (assoc-+ a b c)
```

In questa definizione:

- Nel caso base, quando a è zero, la prova è semplicemente `refl`, poiché zero sommato a qualsiasi numero è uguale a quel numero.
- Nel caso ricorsivo, quando a è il successore di a' , usiamo la funzione di congruenza per applicare la funzione `suc` alla prova che $a' + b + c$ è uguale a $a' + (b + c)$.

3.6 Commutatività

Possiamo anche dimostrare che l'operazione di somma è commutativa utilizzando le proprietà dell'uguaglianza che abbiamo definito.

```

comm-+ : (a b : ℕ) → a + b ≡ b + a
comm-+ zero b = symm-zero-plus b
comm-+ (suc a) b = cong suc (comm-+ a b)

```

💡 Osservazione 3.2

Come notiamo man mano che costruiamo le definizioni di queste proprietà, esse ci aiutano a costruire dimostrazioni più complesse basate su di esse (perché sono "witness" o "testimoni" che ci permettono di affermare certe uguaglianze). La definizione di uguaglianza in Agda e le proprietà che ne derivano (riflessività, simmetria, transitività, congruenza) forniscono una base solida per ragionare sull'uguaglianza tra elementi di tipi diversi. Questi concetti sono fondamentali non solo in Agda, ma anche in molte altre aree della matematica e dell'informatica teorica.

In questa definizione:

- Come parametro, abbiamo due numeri naturali a e b . L'obiettivo è ottenere una dimostrazione che dice che $a + b$ è uguale a $b + a$.
- Nel caso base, quando a è zero, usiamo la funzione `symm-zero-plus` per dimostrare che $b + 0$ è uguale a b .
- Nel caso ricorsivo, quando a è il successore di a' , usiamo la funzione di congruenza per applicare la funzione `suc` alla prova che $a' + b$ è uguale a $b + a'$.

4 Metalogica in Agda

Una cosa interessante di Agda è che possiamo anche modellare concetti di metalogica (quindi possiamo usare Agda per ragionare su sistemi logici stessi), grazie alla sua natura di linguaggio di programmazione e sistema di dimostrazione interattivo. Ad esempio, possiamo definire un semplice sistema logico proposizionale in Agda. Proviamo a modellare alcuni concetti della **logica minimale**, che è una logica proposizionale più debole della logica classica poiché contiene solo l'implicazione e la negazione come connettivi logici. Quest'ultima rifiuta il principio del terzo escluso e l'ex falso quodlibet. Andremo inoltre a definire la deduzione naturale come sistema deduttivo.

4.1 Definizione delle proposizioni e formule

In Agda, possiamo definire un tipo per rappresentare le proposizioni e le formule logiche.

```
open import Data.String

data Form : Set where
  atom : String → Form
  _[⇒]_ : Form → Form → Form
```

Questo non sarebbe un modo molto efficiente per rappresentare le formule logiche, ma per scopi didattici va più che bene. Quindi abbiamo definito un tipo `Form` con due costruttori:

- `atom` rappresenta una proposizione atomica, identificata da una stringa e mi permette di creare proposizioni come "P", "Q", "R", ecc.
- `_ [⇒] _` rappresenta l'implicazione tra due formule logiche.

4.2 Definizione dei contesti

In logica, un contesto è un insieme di assunzioni o ipotesi da cui possiamo dedurre altre proposizioni. In Agda, possiamo rappresentare un contesto come una lista di formule logiche. Anche in questo caso, tocca definirlo in maniera ricorsiva.

```
data Context : Set where
  • : Con
  _▷_ : Con → Form → Con
```

Ricordiamo anche di dare la precedenza all'operatore `▷` e `⇒`:

```
infixr 8 _▷_
infixr 9 _[⇒]_
```

Per evitare di definire ogni volta le variabili e contesti con dei nomi ad ogni definizione (possiamo usare le variabili implicite di Agda):

```
variable Γ Δ : Con
variable ψ φ : Form
```

4.3 Definizione delle regole di deduzione naturale

Dobbiamo ora definire $\vdash$ ovvero il giudizio di deducibilità. Ora possiamo definire le regole di deduzione naturale per la logica minimale (le linee orizzontali chiaramente separano le ipotesi dalle conclusioni, ma non sono codificate all'interno di Agda).

```
infix 5 _⊢_
data _⊢_ : Con → Form → Set where
  -----
  zero : Γ ▷ φ ⊢ φ

  suc : Γ ⊢ ψ →
    -----
    Γ ▷ φ ⊢ ψ

  lam : Γ ▷ φ ⊢ ψ → -- introduzione dell'implicazione
    -----
    Γ ⊢ φ [⇒] ψ

  app : Γ ⊢ φ [⇒] ψ → Γ ⊢ φ → -- modus ponens
    -----
    Γ ⊢ ψ
```

Questa definizione è più complessa perché possiede diversi costruttori:

- **zero**: rappresenta la regola di assunzione, che afferma che se una formula ϕ è nel contesto Γ , allora possiamo dedurre ϕ da Γ .
- **suc**: rappresenta la regola di indebolimento, che afferma che se possiamo dedurre ψ da Γ , allora possiamo anche dedurre ψ da Γ esteso con una nuova formula ϕ .
- **lam**: rappresenta la regola di introduzione dell'implicazione, che afferma che se possiamo dedurre ψ da Γ esteso con ϕ , allora possiamo dedurre l'implicazione $\phi \Rightarrow \psi$ da Γ .

- app: rappresenta la regola di eliminazione dell'implicazione (modus ponens), che afferma che se possiamo dedurre $\phi \Rightarrow \psi$ da Γ e possiamo dedurre ϕ da Γ , allora possiamo dedurre ψ da Γ .

$$\frac{}{\Gamma, \varphi \vdash \varphi} \text{ (zero)}$$

$$\frac{\Gamma \vdash \psi}{\Gamma, \varphi \vdash \psi} \text{ (suc)}$$

$$\frac{\Gamma, \varphi \vdash \psi}{\Gamma \vdash \varphi \Rightarrow \psi} \text{ (lam)}$$

$$\frac{\Gamma \vdash \varphi \Rightarrow \psi \quad \Gamma \vdash \varphi}{\Gamma \vdash \psi} \text{ (app)}$$

4.4 Esempio di prova in Agda

Proviamo a costruire un esempio semplice in Agda usando le regole che abbiamo appena creato. Se io volessi dimostrare che da P implica (banalmente) P e quindi costruire una prova di $P \Rightarrow P$, potrei farlo così:

```
I : • ⊢ (atom "P") [⇒] (atom "P")
-- λ x → x
I = lam zero
```

Sto cercando di provare che dal contesto vuoto (rappresentato da $\bullet$), posso dedurre che P implica P (rappresentato da $(\text{atom "P"}) [\Rightarrow] (\text{atom "P"})$). Sto usando la regola di introduzione dell'implicazione (lam) per costruire questa prova applicando l'assioma che ci permette di "chiudere" la derivazione. Quindi una visualizzazione grafica tramite deduzione naturale sarebbe:

$$\frac{[P]^1}{P \Rightarrow P} (\rightarrow I)^1$$

🔗 Osservazione 4.1

Per semplicità definiamo come assunzione il nostro costruttore zero, mentre suc non fa nulla di più che estendere il contesto con una nuova assunzione che però non viene usata nella dimostrazione (quindi ci permette di creare nuove variabili). Il costruttore lam rappresenta l'introduzione dell'implicazione e app rappresenta l'eliminazione dell'implicazione (modus ponens).

Cerchiamo ora invece di dimostrare:

$$\vdash (P \rightarrow Q \rightarrow R) \rightarrow (Q \rightarrow P \rightarrow R)$$

In Agda, possiamo costruire questa prova come segue:

```
sw : • ⊢ ((atom "P") [⇒] (atom "Q") [⇒] (atom "R"))
[⇒] ((atom "Q") [⇒] (atom "P") [⇒] (atom "R"))
sw = lam (lam (lam (app (app (suc (suc zero)) zero) (suc zero))))
```

Questa definizione può sembrare complessa a prima vista, ma notiamo che stiamo costruendo la prova passo dopo passo utilizzando le regole di deduzione naturale che abbiamo definito. La forma equivalente in deduzione naturale sarebbe:

$$\frac{\frac{\frac{[P \rightarrow (Q \rightarrow R)]^1}{Q \rightarrow R} \quad \frac{[P]^3}{\frac{R}{P \rightarrow R} (\rightarrow I)^3} (\rightarrow E) \quad \frac{[Q]^2}{Q \rightarrow (P \rightarrow R)} (\rightarrow E)}{\frac{Q \rightarrow (P \rightarrow R)}{(P \rightarrow Q \rightarrow R) \rightarrow (Q \rightarrow P \rightarrow R)} (\rightarrow I)^2} (\rightarrow I)^1$$

🔗 Osservazione 4.2

Notiamo come abbiamo applicato la regola di introduzione dell'implicazione (1am) tre volte e la regola di eliminazione dell'implicazione (app) due volte, proprio come nella derivazione in Agda.

Ora che abbiamo visto la deduzione naturale in maniera grafica, proviamo a srotolare quello che abbiamo fatto invece in agda. Il modo intuitivo di costruire la prova in Agda e provarla a guardarla con un approccio bottom-up (chi ha esperienza con questo tipo di esercizi, sa che è il modo più semplice per affrontarli):

- La tesi è di tipo $(P \rightarrow Q \rightarrow R) \rightarrow (Q \rightarrow P \rightarrow R)$, quindi per costruire una prova di questo tipo, dobbiamo sicuramente introdurre un'ipotesi $P \rightarrow Q \rightarrow R$ nelle assunzioni. Usiamo quindi 1am per introdurre questa ipotesi.
- Ora, la tesi diventa $Q \rightarrow P \rightarrow R$, quindi dobbiamo introdurre un'altra ipotesi Q nelle assunzioni. Usiamo di nuovo 1am per introdurre questa ipotesi.
- Ora, la tesi diventa $P \rightarrow R$, quindi dobbiamo introdurre un'ultima ipotesi P nelle assunzioni. Usiamo ancora 1am per introdurre questa ipotesi.
- A questo punto abbiamo applicato 3 volte 1am, e la tesi diventa R .
- Possiamo ottenere R applicando Modus Ponens a $Q \rightarrow R$ e Q .
- A sua volta $Q \rightarrow R$ può essere ottenuto applicando Modus Ponens a $P \rightarrow Q \rightarrow R$ e P .
- Quindi alla fine le assunzioni che abbiamo chiuso sono proprio:
 - $P \rightarrow Q \rightarrow R$ (chiusa con la prima 1am),
 - Q (chiusa con la seconda 1am),
 - P (chiusa con la terza 1am).